

Random Fourier Feature Kernel Recursive Least Squares

Zhengda Qin, Badong Chen, Nanning Zheng

Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University, Xi'an, China
chenbd@mail.xjtu.edu.cn

Abstract—In this paper, we investigate the nonlinear, finite dimensional and data independent random Fourier feature expansions that can approximate the popular Gaussian kernel. With recursive least squares algorithm, we develop the *Random Fourier Feature Recursive Least Squares* algorithm (RFF-RLS), which shows significant performance improvements in simulations when compared with several other online kernel learning algorithms such as *Kernel Least Mean Square* (KLMS) and *Kernel Recursive Least Squares* (KRLS). Our results confirm that the RFF-RLS can achieve desirable performance with low computational cost. As for the random Fourier features, the randomization generally results in redundancy. We use an algorithm, namely, *Vector Quantization with Information Theoretic Learning* (VQIT) to decrease the dictionary size. The resulting sparse dictionary can match the original data distribution well. The RFF-RLS with VQIT can outperform the RFF-RLS without VQIT.

1. Introduction

In machine learning, kernel methods are a powerful and straightforward way of mapping the data to a high dimensional kernel space to deal with nonlinearities [1]. There are many successful algorithms with "kernel trick" such as kernel-SVM [2], kernel-PCA [3] and *Kernel Adaptive Filters* (KAFs) [4]. The KAFs are very attractive because of their desirable features such as convexity, universal approximation, and online learning manner [5], [6]. The kernel adaptive filters, which create a growing *Linear-In-the-Parameter* (LIP) nonlinear model in the input space, are developed by implementing the well-established linear adaptive filters in kernel space. So far, many KAF algorithms have been proposed such as *Kernel Least Mean Square* (KLMS), *Kernel Affine Projection* (KAP) and *Kernel Recursive Least Squares* (KRLS) and so on [4], [7].

The KAF algorithms have a bottleneck of computational complexity, requiring linear or superlinear time and space respect to the sample number n . Because of such disadvantage, these methods cannot be efficiently implemented for large-scale applications in practice. Existing approaches to address this problem include *Vector Quantization* (VQ) [8], [9], [10], *Novelty Criterion* (NC), *Approximate Linear Dependency* (ALD) [4], and many others. These approaches sparsify the dictionary by carefully weeding out samples

which are not important. However, the complexity still increases over time in general.

Two different well-known approaches for generating a dictionary with fixed size are Nyström method [11] and *Random Fourier Feature* (RFF). Nyström method was first introduced to the machine learning community using uniform sampling without replacement [12]. The Nyström method operates on a subset of samples and generates low-rank matrix approximation of the original kernel matrix $\mathbf{K}$. There are several theoretical results about Nyström method which bound the error in approximating the kernel matrix $\mathbf{K}$ [13]. RFF method was first proposed by Rahimi and Recht [14], [15], which represents the data points by using a finite dimensional space to approximate the kernel space. The theoretical foundation for random Fourier transformation is *Bochner's Theorem* [16], which suggests that a translation invariant or shift invariant kernel $\kappa(\mathbf{x}, \mathbf{y}) = \kappa(\mathbf{x} - \mathbf{y})$ on $\mathbb{R}^d$ is positive definite if and only if κ is a Fourier transformation of non-negative measure. The generalization error bound was analyzed for kernel learning based on random Fourier features [17]. Yang et al. compared Nyström method and random Fourier feature and showed that Nyström method is better than random Fourier feature when there is a large eigengap of kernel matrix [18]. However, random Fourier features are data independent basis functions which have significant strengths for online learning especially for non-stationary situations. In this paper, we mainly use the RFF to approximate the kernel function and propose an efficient online learning algorithm.

Singh et al. used the random Fourier feature with a gradient optimization in KLMS to develop an algorithm which is called RFF-KLMS [19]. The RFF-KLMS has constant time and memory complexity like a linear method. In this work, we use the *Exponentially-Weighted Recursive Least Squares* algorithm (EW-RLS) [20] to optimize the weights of RFF bases, which can significantly improve the learning performance while keeping constant time and memory complexity. Simulation results show that the proposed *Random Fourier Feature Recursive Least* algorithm (RFF-RLS) with very small computational cost can outperform the original KLMS and can also beat existing sparsified KRLS algorithms [4], [21]. The randomization is likely to draw some outliers far from the distribution centre, which may result in a large approximation error. At the same time, the randomization also leads to redundancy in the

dictionary. To overcome the outliers and redundancy caused by the randomization, we apply the *Vector Quantization with Information Theoretic Learning* (VQIT) [22] to reduce the dictionary size of RFF. This method is able to preserve data distribution, while discarding the outliers and redundancy in dictionary. With VQIT, the performance of RFF-RLS can be further improved.

The rest of the paper is organized as follows. In section 2, we briefly introduce the *Exponentially-Weighted Kernel Recursive Least Squares* (EW-KRLS) which is a non-linear extension to the EW-RLS algorithm. In section 3, we propose the RFF-RLS algorithm, which is a random approximation of EW-KRLS, and compare the computational complexity of the RFF-RLS with several relevant algorithms. In section 4, we describe the VQIT and use it to quantize the randomly generated dictionary. In the section 5, we present simulation results to demonstrate the desirable performance of RFF-RLS. Finally, in section 5, we give the conclusion.

2. Kernel Recursive Least Squares

Consider N pairs of training data $\{\mathbf{x}(i), d(i)\}_{i=1}^N$, with $\mathbf{x}(i)$ being the input vector at time i , and $d(i)$ the desired signal. Let $f_i(\cdot)$ be the learned function at time i . When a new sample is available at time i , this function makes a prediction $\hat{y}(i) = f_i(\mathbf{x}(i))$. The goal of online learning is to update the prediction function sequentially by minimizing a cost J . With kernel methods, the prediction function $f(\cdot)$ usually takes the form:

$$f(\mathbf{x}) = \langle \mathbf{w}, \varphi(\mathbf{x}) \rangle_{\mathbf{H}} = \mathbf{w}^T \varphi(\mathbf{x}) \quad (1)$$

where $\varphi(\mathbf{x})$ is a nonlinear mapping induced by a Mercer kernel $\kappa(\mathbf{x}, \mathbf{x}')$ that maps the input $\mathbf{x}$ to a high-dimensional feature space $\mathbf{H}$, $\mathbf{w}$ is a weight vector in $\mathbf{H}$ that needs to be estimated, and $\langle \cdot, \cdot \rangle_{\mathbf{H}}$ denotes the inner product in $\mathbf{H}$. Then, updating the prediction function amounts to updating the weight vector $\mathbf{w}$.

The KRLS algorithm generally operates by minimizing the following cost:

$$J(i) = \sum_{j=1}^i \beta^{i-j} \left[d(j) - \mathbf{w}(i-1)^T \varphi(\mathbf{x}(j)) \right]^2 + \beta^i \lambda \|\mathbf{w}(i-1)\|^2 \quad (2)$$

where β denotes the forgetting parameter and λ stands for the regularization parameter. The weight vector $\mathbf{w}$ can be obtained by [20]:

$$\begin{aligned} \mathbf{w}(i) &= \left[\lambda \beta^i \mathbf{I} + \Phi(i) \mathbf{B}(i) \Phi(i)^T \right]^{-1} \Phi(i) \mathbf{B}(i) \mathbf{d}(i) \\ &= \Phi(i) \left[\lambda \beta^i \mathbf{B}(i)^{-1} + \Phi(i)^T \Phi(i) \right]^{-1} \mathbf{d}(i) \end{aligned} \quad (3)$$

where $\varphi(\mathbf{x}(j)) = \varphi(i)$, $\Phi(i) = [\varphi(1), \varphi(2), \dots, \varphi(i)]^T$, $\mathbf{d}(i) = [d(1), d(2), \dots, d(i)]^T$ and $\mathbf{B}(i) = \text{diag}(\beta^{i-1}, \beta^{i-2}, \dots, 1)$. According to the representing

theory, one can use a linear combination $\Phi(i) \mathbf{a}(i)$ to represent the weight vector $\mathbf{w}$, and this leads to:

$$\mathbf{a}(i) = \left[\lambda \beta^i \mathbf{B}(i) + \Phi(i)^T \Phi(i) \right]^{-1} \mathbf{d}(i) \quad (4)$$

Let $\mathbf{Q}(i) = \left[\lambda \beta^i \mathbf{B}(i) + \Phi(i)^T \Phi(i) \right]^{-1}$. Using the block matrix inversion identity, one can obtain the following update equation for $\mathbf{Q}$:

$$\mathbf{Q}(i) = r(i)^{-1} \begin{bmatrix} \mathbf{Q}(i-1) r(i) + \mathbf{Z}(i) \mathbf{Z}(i)^T & -\mathbf{Z}(i) \\ -\mathbf{Z}(i)^T & 1 \end{bmatrix} \quad (5)$$

with

$$\mathbf{h}(i) = \Phi(i-1)^T \varphi(i) = [\kappa(\mathbf{x}(1), \mathbf{x}(i)), \dots, \kappa(\mathbf{x}(i-1), \mathbf{x}(i))] \quad (6)$$

$$\mathbf{Z}(i) = \mathbf{Q}(i-1) \mathbf{h}(i) \quad (7)$$

$$r(i) = \lambda \beta^i + \varphi(i)^T \varphi(i) - \mathbf{Z}(i)^T \mathbf{h}(i) \quad (8)$$

Then the coefficient vector $\mathbf{a}(i)$ can be updated recursively as

$$\begin{aligned} \mathbf{a}(i) &= \mathbf{Q}(i) \mathbf{d}(i) \\ &= \begin{bmatrix} \mathbf{a}(i-1) - \mathbf{Z}(i) r(i)^{-1} e(i) \\ r(i)^{-1} e(i) \end{bmatrix} \end{aligned} \quad (9)$$

where $e(i) = d(i) - \mathbf{h}(i)^T \mathbf{a}(i-1)$. The above algorithm is called the EW-KRLS algorithm. In this work, without explicit mention, the kernel function in KRLS is the Gaussian kernel:

$$\kappa(\mathbf{x}, \mathbf{y}) = \exp \left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2} \right) \quad (10)$$

where $\sigma > 0$ is the kernel bandwidth. With a Gaussian kernel, the KRLS produces a growing RBF network with growing computational complexity. In order to reduce the model complexity and simplify the computation, one can use some sparsification methods, such as VQ and NC [4], [8], to prune the insignificant nodes. In the next section, we use the RFF to overcome the growing sum problem of KRLS and develop the RFF-RLS algorithm.

3. Random Fourier Feature Recursive Least Squares Algorithm

Using the kernel-trick to map the original data to a high-dimensional Hilbert space and using the RLS algorithm to handle the transformed data yield the KRLS. Random Fourier feature is a *finite* dimensional mapping $\mathbf{z} : \mathbb{R}^d \rightarrow \mathbb{R}^p$ that can approximate the Mercer kernel by the inner product between a pair of transformed points [14]:

$$\kappa(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle \approx \mathbf{z}(\mathbf{x})^T \mathbf{z}(\mathbf{y}) \quad (11)$$

KAF algorithms like KRLS require learning the function $f(\cdot) = \sum_{j=1}^i a(j) \kappa(\mathbf{x}(j), \cdot)$, which needs a lot of data to train and the data need to be stationary in general. However,

the superlinear growth complexity makes KRLS not acceptable for large datasets based learning in practice. With the random Fourier feature, one can compute $\hat{f}(\mathbf{x}) = \mathbf{w}^T \mathbf{z}(\mathbf{x})$ and learn the hyperplane $\mathbf{w}$ in a finite dimensional space. In this way, the random feature $\mathbf{z}(\mathbf{x})$ makes the KRLS like a linear model, which requires fixed time and memory at each iteration.

Studies along this direction date back to a theorem called *Bochner's Theorem* [16]: A kernel $\kappa(\mathbf{x}, \mathbf{y}) = \kappa(\mathbf{x} - \mathbf{y})$ on $\mathbb{R}^d$ is positive definite if and only if κ is the Fourier transform of a non-negative measure. The inner product of finite dimensional random features was proposed by Rahimi et al. who subtly convert the Fourier transform $p(\omega)$ to a probability distribution. Defining $z_\omega(\mathbf{x}) = e^{j\omega^T \mathbf{x}}$, the Bochner's theorem guarantees the equation:

$$\kappa(\mathbf{x} - \mathbf{y}) = \int_{\mathbb{R}^d} p(\omega) e^{j\omega^T (\mathbf{x} - \mathbf{y})} d\omega = E_\omega [z_\omega(\mathbf{x})^* z_\omega(\mathbf{y})] \quad (12)$$

The middle part in (12) represents not only inverse Fourier transformation but also the expectation of $z_\omega(\mathbf{x})^* z_\omega(\mathbf{y})$. So it is an unbiased estimated of $\kappa(\mathbf{x} - \mathbf{y})$ when finite samples of ω are drawn from $p(\omega)$. The Gaussian kernel function is real-valued, so the integrand $e^{j\omega^T (\mathbf{x} - \mathbf{y})}$ can be replaced by $\cos \omega^T (\mathbf{x} - \mathbf{y})$. Thus, $z_\omega(\mathbf{x}) = [\cos \omega^T \mathbf{x}, \sin \omega^T \mathbf{x}]$ provides a real-valued feature mapping, where ω is randomly drawn from $p(\omega) \sim N(0, \mathbf{I}_d / \sigma^2)$. Another equivalent mapping is $z_\omega(\mathbf{x}) = \sqrt{2} \cos(\omega^T \mathbf{x} + b)$, where ω is also drawn from $p(\omega) \sim N(0, \mathbf{I}_d / \sigma^2)$ and b is drawn uniformly from $[0, 2\pi]$ [14]. Both mappings satisfy the condition $E_\omega [z_\omega(\mathbf{x})^* z_\omega(\mathbf{y})] = \kappa(\mathbf{x} - \mathbf{y})$. The average of D -dimensional inner products in (13) is the approximation of kernel evaluation $\kappa(\mathbf{x} - \mathbf{y})$:

$$\mathbf{z}(\mathbf{x})^T \mathbf{z}(\mathbf{y}) = \frac{1}{D} \sum_{j=1}^D z_{\omega_j}(\mathbf{x})^* z_{\omega_j}(\mathbf{y}) \quad (13)$$

where $\mathbf{z}(\mathbf{x}) = [z_{\omega_1}(\mathbf{x}), z_{\omega_2}(\mathbf{x}), \dots, z_{\omega_D}(\mathbf{x})]^T$ is the D -dimensional *random Fourier feature* (RFF) of the input vector $\mathbf{x}$. In (13), D is a predefined parameter that controls the dictionary size of RFF and the quality of the approximation. The approximation accuracy improves exponentially fast with respect to this parameter [14].

In summary, the KAFs with the Gaussian kernel train the filter in a high dimensional (possibly infinite) Hilbert space, while RFF provides us a way to map the input vector into a finite (much lower) dimensional space in which the inner products can approximate the kernel evaluation well. Dealing with the data in a finite space, random feature mappings efficiently decreases the computational cost. All the parameters except D are randomized in RFF mappings without having the information of data in advance. Randomization is an old principle used in learning, the data independent characteristic makes it very suitable for online learning.

Since this transformed space is finite dimensional, one can learn the hyperplane $\mathbf{w}$ by using linear methods and

approximate the prediction function $f(\mathbf{x})$ of KRLS by computing $\hat{f}(\mathbf{x}) = \mathbf{w}^T \mathbf{z}(\mathbf{x})$. As we mentioned before, we use the EW-RLS algorithm with high convergence speed to learn the hyperplane $\mathbf{w}$ and propose *RFF recursive least squares algorithm* (RFF-RLS). The proposed RFF-RLS algorithm is described below:

Algorithm 1 RFF-RLS

- 1: **Inpu**t: Sequential training data $\{\mathbf{x}(i), d(i)\}_{i=1}^N$, Gaussian kernel width σ , regularization factor λ , forgetting factor β and RFF dimension D .
 - 2: **Draw**: i.i.d. $\{\omega_i\}_{i=1}^D$ from $N(0, \mathbf{I}_d / \sigma^2)$, where d is the input space dimension.
 - 3: **Draw**: i.i.d. $\{b_i\}_{i=1}^D$ from *Uniform* $[0, 2\pi]$.
 - 4: **initialize**: $\mathbf{w}(0) = 0$, $\mathbf{P}(0) = \lambda^{-1} \mathbf{I}_D$
 - 5: **for** $i = 1$ to N **do**
 - 6: Compute random Fourier feature vector $\mathbf{z}(\mathbf{x}(i)) = [z_{\omega_1}(\mathbf{x}(i)), z_{\omega_2}(\mathbf{x}(i)), z_{\omega_3}(\mathbf{x}(i)), \dots, z_{\omega_D}(\mathbf{x}(i))]^T$, where $z_\omega(\mathbf{x}(i)) = \sqrt{2} \cos(\omega^T \mathbf{x}(i) + b)$;
 $e(i) = d(i) - \langle \mathbf{w}(i), \mathbf{z}(\mathbf{x}(i)) \rangle$;
 $\mathbf{s} = \mathbf{P} \mathbf{z}(\mathbf{x}(i))$;
 $\mathbf{k} = \mathbf{s} / (\beta + \mathbf{z}(\mathbf{x}(i))^T \mathbf{s})$;
 $\mathbf{w}(i+1) = \mathbf{w}(i) + \mathbf{k} e(i)$;
 $\mathbf{P}(i+1) = \mathbf{P}(i) - \mathbf{k} \mathbf{s}^T$
 - 7: **end for**
-

In addition, the RFF mappings provides us a way of fast learning. KLMS evaluate the prediction function $f(\cdot) = \sum_{j=1}^i a(j) \kappa(\mathbf{x}(j), \cdot)$, requiring $O(id)$ operations and storage. KRLS, performing much better than KLMS, needs $O(i^2 d)$ operations and storage respectively. With sparsification, the complexity of KRLS will be reduced to $O(D_i^2)$ where D_i is the dictionary size in time i . With some linear methods such as RLS and *Least Mean Square* (LMS) algorithm, the RFF expansions can realize a constant time and space complexity. The approximation error bound provided in [14] shows that the RFF is able to uniformly approximate the shift-invariant kernels $\kappa(\mathbf{x} - \mathbf{y})$ within a small error ϵ when $D = O(d \epsilon^{-2} \log \frac{1}{\epsilon^{-2}})$ dimensions. Although the RFF-RLS algorithm requires $O(D^2 + d)$ operations and storage, while RFF-KLMS requires only $O(D + d)$, it's not a heavy computational burden when D is a small value. At the same time, the RLS can tremendously improve the performance. Compared with RFF-KLMS, KLMS and several sparsified KRLS algorithms, RFF-RLS shows excellent results even with a very small value of D .

4. Vector Quantization with Information Theoretic Learning

The randomization is likely to generate some points far away from the distribution centre, which suffers the performance. At the same time, the randomization also leads to redundancy in the dictionary. In order to overcome the outliers and redundancy caused by the randomization, we

use an algorithm for *Vector Quantization using Information Theoretic Learning* (VQIT) to represent ω instead of drawing it randomly. This method can cut the outliers and preserve the characteristic of data distribution. Actually, VQIT is a density matching method which minimize the free energy of the system [22]. With this algorithm, we can generate a large set of ω in Gaussian distribution first and use a small set of substitution $\hat{\omega}$ to represent the large one.

Given a large set of data $\{\omega_i\}_{i=1}^D$, VQIT generates a smaller set $\{\hat{\omega}_j\}_{j=1}^M$ with $M < D$ by minimizing the following cost function [22]:

$$J(\hat{\omega}) = \log \int f^2(\omega) d\omega - 2 \log \int f(\omega) g(\hat{\omega}) d\omega + \log \int g^2(\hat{\omega}) d\omega \quad (14)$$

where $f(\omega)$ denotes the distribution function of data ω , and $g(\hat{\omega})$ denotes the distribution function of a smaller set of *Processing Elements* (PEs) $\hat{\omega}$. Here, $V = \int g^2(\hat{\omega}) d\omega$ is the information potential of PEs and $C = \int f(\omega) g(\hat{\omega}) d\omega$ is the cross-information potential between the densities of the PEs and the data [23]. The cost function J is actually the divergence between a smaller set of PEs density distribution and data distribution. We estimate the distributions by Parzen estimator and yield:

$$C = \int \hat{f}(\omega) \hat{g}(\hat{\omega}) d\omega = \frac{1}{MD} \sum_{i=1}^D \sum_{j=1}^M \kappa_{\sigma_a}(\omega_i - \hat{\omega}_j) \quad (15)$$

$$V = \int \hat{g}^2(\hat{\omega}) d\omega = \frac{1}{M^2} \sum_{i=1}^M \sum_{j=1}^M \kappa_{\sigma_g}(\hat{\omega}_i - \hat{\omega}_j) \quad (16)$$

where $\kappa_{\sigma}(\cdot)$ is the Gaussian kernel with bandwidth σ . Then, one can update the locations of PEs by gradient descent methods. According to the above cost function, we adopt the following update rule for the point k -th PEs [22]

$$\hat{\omega}_k(n+1) = \hat{\omega}_k(n) - \eta \left(\frac{\Delta V}{V} - 2 \frac{\Delta C}{C} \right) \quad (17)$$

where η is the step size parameter, and the gradients ΔC and ΔV are:

$$\Delta C(\hat{\omega}_k) = \frac{1}{MD} \sum_{i=1}^D \kappa_{\sigma_a}(\omega_i - \hat{\omega}_k) \sigma_a^{-2} (\omega_i - \hat{\omega}_k) \quad (18)$$

$$\Delta V(\hat{\omega}_k) = \frac{1}{M^2} \sum_{j=1}^M \kappa_{\sigma_g}(\hat{\omega}_j - \hat{\omega}_k) \sigma_g^{-2} (\hat{\omega}_j - \hat{\omega}_k) \quad (19)$$

With the above VQIT method, one can obtain a much smaller set of $\hat{\omega}$ before training. The RFF-RLS with VQIT further improves the performance, since the simplified dictionary can match the original distribution well and remove the outliers.

5. Simulations

5.1. Non-Stationary Regression

Consider the following nonlinear time varying system [19]:

$$x(i) \sim \text{Uniform}[0, 1], \quad i = 1, 2, \dots, 600. \quad (20)$$

$$y(i) = \begin{cases} \sin(10x(i)) + v(i) & \text{if } i \leq 300 \\ \sin(12x(i)) + v(i) & \text{if } i > 300 \end{cases} \quad (21)$$

where $v(i) \sim \text{Uniform}[0, 0.5]$ is i.i.d. observation noise, independent of $x(i)$. Fig. 1 shows the generated samples from the above system. Obviously, this is a nonlinear system and the signal changes at $i = 300$.

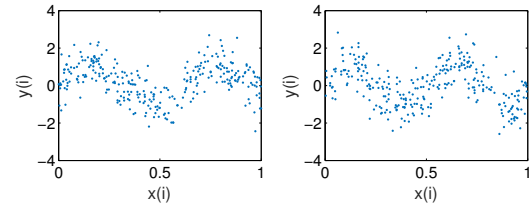


Figure 1. Samples generated by (20) and (21). (a): $i \leq 300$, (b): $i > 300$.

All the samples are generated by (20) and (21). The training set includes 600 samples and the testing set contains 100 samples. We use the training set to train the KLMS, RFF-KLMS and the proposed RFF-RLS algorithm, and test the performance at each iteration with the testing set. We compute the *Mean Square Error* (MSE) at every iteration and plot the average learning curves of 50 independent Monte Carlo runs. The Gaussian kernel width σ is set at 0.1.

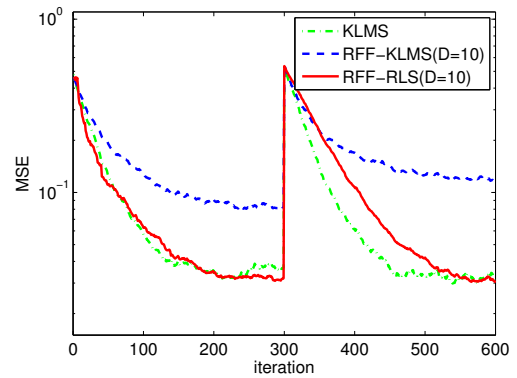


Figure 2. Convergence curves of KLMS, RFF-KLMS ($D=10$) and RFF-RLS ($D=10$)

We compare the performance of KLMS, RFF-KLMS and RFF-RLS when $D = 10$ in Fig.2. The stepsize parameter for KLMS and RFF-KLMS are 0.1 and 0.2 respectively. The regularization factor λ and forgetting factor β for RFF-RLS are 1.5 and 0.99. Fig.3 shows performance of KLMS, RFF-KLMS and RFF-RLS when $D = 50$. In this comparison, the stepsize parameter for KLMS and RFF-KLMS are

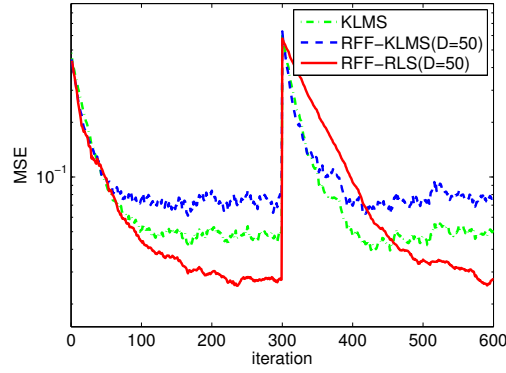


Figure 3. Convergence curves of KLMS, RFF-KLMS (D=50) and RFF-RLS (D=50)

0.15 and 0.5 respectively. The regularization factor λ and forgetting factor β for RFF-RLS are 10 and 0.99. Appropriate values for both regularization factor and forgetting factor are easy to find by cross validation. Clearly, the RFF-RLS is much better than RFF-KLMS and it outperforms the KLMS when $D = 50$. For RFF-RLS and RFF-KLMS, their performance improves when choosing a larger value of D . For a fixed pair of points $\mathbf{x}$ and $\mathbf{y}$, Hoeffding's inequality guarantees exponentially fast convergence with respect to D between $z(\mathbf{x})^T z(\mathbf{y})$ and $\kappa(\mathbf{x}, \mathbf{y})$ [14]. Even choosing a small value of D such as 10, the RFF-RLS achieves good performance since the testing MSE is almost the same as KLMS. The computational complexity of RFF-KLMS is just a little bigger than RFF-RLS in this simulation, which is much smaller than KLMS, we show the CPU execution time (second) requirement in each iteration in Fig. 4.

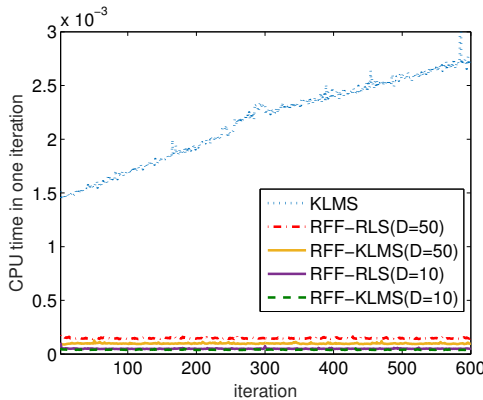


Figure 4. CPU execution time for training and testing at each iteration for KLMS, RFF-KLMS (D=10,50) and RFF-RLS (D=10,50)

5.2. Mackey-Glass Chaotic Time Series Prediction

In this simulation, we compare several sparsification approaches in KRLS including *novelty criterion* (KRLS-NC), *approximate linear dependency* (KRLS-ALD), *Vector quantization* (QKRLS) and *enhanced novelty criterion* (KRLS-ENC) with the proposed RFF-RLS algorithm. We use the Mackey-Glass chaotic time series as a benchmark

data set to compare these algorithms. The previous 7 point $\mathbf{x}(i) = [x(i-7), x(i-6), \dots, x(i-1)]^T$ are used to predict the present one $x(i)$. The training set includes 1000 samples and another 100 as the testing set. The train samples are corrupted by i.i.d. an additive white Gaussian Noise with zero mean and variance 0.001. The Gaussian kernel width parameter $\sigma = \frac{1}{\sqrt{2}}$. We choose 20 values of threshold δ_1 uniformly in $[0.05, 0.2]$ and 20 values of threshold δ_2 uniformly in $[0.05, 0.2]$ for NC. We choose 100 values of threshold δ uniformly in $[0.04, 0.3]$ for ALD. We choose 20 values of threshold δ_1 uniformly in $[0.04, 0.2]$ and 20 values of δ_2 uniformly in $[0.05, 0.2]$ for ENC. As for QKRLS, we choose 40 quantization radii ϵ_q uniformly in $[0.16, 0.36]$ [4]. At last, we choose 100 values D uniformly in $[30, 430]$ uniformly. The regularization parameter is set 0.001 for KRLS-NC, RLS-ALD and RLS-ENC. The regularization parameter are set 0.01 and 1 respectively for QKRLS and RFF-RLS. The forgetting parameter is 1 for QKRLS and RFF-RLS. For each possible combination of parameter, we train the KRLS algorithm with corresponding sparsification approach, and record the network size and the testing MSE. Each simulation result is a point in fig. 5.

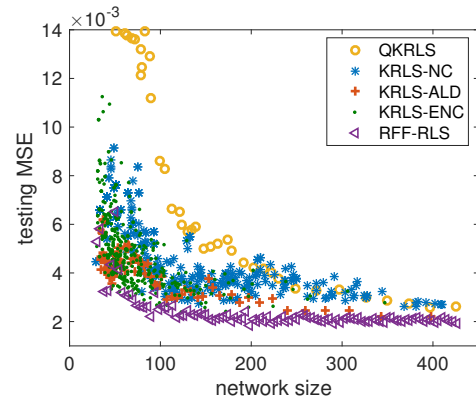


Figure 5. Performance of different KRLS algorithm

The computational complexity of these algorithm are almost the same when their network size are the same. Result shows that the RFF-RLS with different network size always beats the other quantized KRLS algorithms. Both these two simulations show the advantage of RFF-RLS, which suggests that RFF can achieve excellent result with a small computational complexity when given a small value of D . This advantage makes RFF-RLS appropriate for online learning in actual occasions. We also compare quantized KRLS with RFF-RLS in the real-world data set like Internet traffic prediction data set and have some satisfactory results too.

Next, we compare the performance of RFF-RLS and RFF-RLS with VQIT when their dictionary size D are 15. Using VQIT, we obtain 15 points of $\hat{\omega}$ from 1000 points which is generated randomly from Gaussian distribution where σ_a^2 and σ_g^2 are 0.32 and 0.16 respectively. Next, we generate 15 points of ω from Gaussian distribution directly. In this simulation, we also use Mackey-Glass chaotic time series data set to compare the performance of RFF-RLS

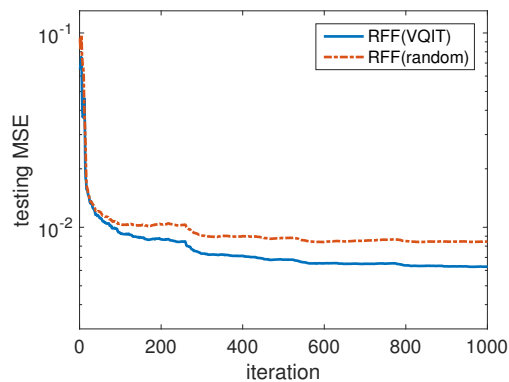


Figure 6. Convergence curves of RFF-RLS with a small dictionary drawn by randomization versus VQIT

computed by these two different mappings. All the data are the same as the last one. The Gaussian kernel width parameter σ is also $\frac{1}{\sqrt{2}}$. The regularization parameter and the forgetting parameter are all 1 like the last example. Fig. 6 plots average learning curves of 100 Monte Carlo independent tests, which shows the improvements of performance brings by VQIT.

6. Conclusion

Random Fourier Feature (RFF) is a randomization approach for simplifying the kernel learning methods, which is suitable for online learning. In this paper, we apply it to approximate the Gaussian kernel and develop an efficient online learning algorithm called RFF-RLS. The new algorithm is derived by combining the RFF and the *Exponentially-Weighted Recursive Least Squares* (EW-RLS) algorithm. To further reduce the computational complexity, we use the *Vector Quantization with Information Theoretic Learning* (VQIT) to remove the outliers and redundancy in the randomly generated dictionary, while remaining almost the same distribution. Simulation results demonstrate that the proposed method can achieve superior performance.

Acknowledgments

This work was supported by 973 Program (No. 2015CB351703) and National Natural Science Foundation of China (No. 61372152, No.91648208).

References

- [1] B. Scholkopf and A. J. Smola, *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.
- [2] C. Cortes and V. Vapnik, "Support-vector networks," *Machine learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [3] B. Scholkopf, A. Smola, and K.-R. Müller, "Nonlinear component analysis as a kernel eigenvalue problem," *Neural computation*, vol. 10, no. 5, pp. 1299–1319, 1998.
- [4] W. Liu, J. C. Principe, and S. Haykin, *Kernel adaptive filtering: a comprehensive introduction*. John Wiley & Sons, 2011, vol. 57.
- [5] J. Kivinen, A. J. Smola, and R. C. Williamson, "Online learning with kernels," *IEEE transactions on signal processing*, vol. 52, no. 8, pp. 2165–2176, 2004.
- [6] C. Richard, J. C. M. Bermudez, and P. Honeine, "Online prediction of time series data with kernels," *IEEE Transactions on Signal Processing*, vol. 57, no. 3, pp. 1058–1067, 2009.
- [7] Y. Engel, S. Mannor, and R. Meir, "The kernel recursive least-squares algorithm," *IEEE Transactions on signal processing*, vol. 52, no. 8, pp. 2275–2285, 2004.
- [8] B. Chen, S. Zhao, P. Zhu, and J. C. Principe, "Quantized kernel least mean square algorithm," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 23, no. 1, pp. 22–32, 2012.
- [9] S. Nan, L. Sun, B. Chen, Z. Lin, and K.-A. Toh, "Density-dependent quantized least squares support vector machine for large data sets," *IEEE transactions on neural networks and learning systems*, vol. 28, no. 1, pp. 94–106, 2017.
- [10] X. Xu, H. Qu, J. Zhao, X. Yang, and B. Chen, "Quantised kernel least mean square with desired signal smoothing," *Electronics Letters*, vol. 51, no. 18, pp. 1457–1459, 2015.
- [11] A. Golts and M. Elad, "Linearized kernel dictionary learning," *IEEE Journal of Selected Topics in Signal Processing*, vol. 10, no. 4, pp. 726–739, 2016.
- [12] C. Williams and M. Seeger, "Using the nyström method to speed up kernel machines," in *Proceedings of the 14th annual conference on neural information processing systems*, no. EPFL-CONF-161322, 2001, pp. 682–688.
- [13] P. Drineas and M. W. Mahoney, "On the nyström method for approximating a gram matrix for improved kernel-based learning," *Journal of Machine Learning Research*, vol. 6, no. Dec, pp. 2153–2175, 2005.
- [14] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in neural information processing systems*, 2007, pp. 1177–1184.
- [15] —, "Uniform approximation of functions with random bases," in *Communication, Control, and Computing, 2008 46th Annual Allerton Conference on*. IEEE, 2008, pp. 555–561.
- [16] M. Reed and B. Simon, "Method of modern mathematical physics ii," *Fourier analysis*, 1975.
- [17] A. Rahimi and B. Recht, "Weighted sums of random kitchen sinks: Replacing minimization with randomization in learning," in *Advances in neural information processing systems*, 2009, pp. 1313–1320.
- [18] T. Yang, Y.-F. Li, M. Mahdavi, R. Jin, and Z.-H. Zhou, "Nyström method vs random fourier features: A theoretical and empirical comparison," in *Advances in neural information processing systems*, 2012, pp. 476–484.
- [19] A. Singh, N. Ahuja, and P. Moulin, "Online learning with kernels: Overcoming the growing sum problem," in *2012 IEEE International Workshop on Machine Learning for Signal Processing*. IEEE, 2012, pp. 1–6.
- [20] S. S. Haykin, *Adaptive filter theory*. Pearson Education India, 2008.
- [21] B. Chen, S. Zhao, P. Zhu, and J. C. Principe, "Quantized kernel recursive least squares algorithm," *IEEE transactions on neural networks and learning systems*, vol. 24, no. 9, pp. 1484–1491, 2013.
- [22] T. Lehn-Schiøler, A. Hegde, D. Erdogmus, and J. C. Principe, "Vector quantization using information theoretic concepts," *Natural Computing*, vol. 4, no. 1, pp. 39–51, 2005.
- [23] J. C. Principe, D. Xu, Q. Zhao, and J. W. Fisher Iii, "Learning from examples with information theoretic criteria," *Journal of VLSI signal processing systems for signal, image and video technology*, vol. 26, no. 1-2, pp. 61–77, 2000.